

Reviewer Pack — Minimal Order of p-Adic Divergence and Communication Complex...

Entry e29119440ded · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 04:11:41 UTC

Minimal Order of p-Adic Divergence and Communication Complexity

Entry ID: e29119440ded **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 04:11:41 UTC

1. Conjecture

Field A (mathematical branch): p-Adic Analysis **Field B** (complexity object): Communication Complexity

Statement:

For any given Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the minimal p-adic divergence ($\min_d(f)$) of its associated boolean circuit c_f is linearly correlated with its communication complexity $c(f)$, such that $\min_d(f) = \Theta(c(f))$.

Rationale (proposer's reasoning):

p-Adic analysis provides a framework for studying arithmetic properties of functions. By exploring the p-adic divergence, which measures the distance between two functions in the p-adic metric space, we may uncover new relationships with communication complexity, a measure of the amount of information that needs to be communicated between two parties to compute a function.

Taxonomy category: P-ADIC_ANALYSIS_COMMUNICATION_COMPLEXITY_CORRELATION (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 32669b1b5329fcf5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the generated Boolean functions $f: \{0,1\}^n \rightarrow \{0,1\}$ ($n \leq 40$) have a p-adic divergence $\min_d(f)$ that correlates with their communication complexity $c(f)$, with an absolute correlation coefficient ≥ 0.7 and mean difference ≤ 3 across all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_boolean_function(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def construct_circuit(f):
    n = int(math.log2(len(f)))
    circuit = []
    for i in range(n):
        layer = []
        for j in range(2**(n-i-1)):
            if f[2*j] == f[2*j+1]:
                layer.append((f[2*j], 'AND'))
            else:
                layer.append((f[2*j], 'OR'))
        circuit.append(layer)
    return circuit

def p_adic_divergence(f):
    n = int(math.log2(len(f)))
    count = 0
    for i in range(1, n+1):
        for j in range(2**(n-i)):
            if f[2*j] != f[2*j+1]:
                count += 1
    return Fraction(count, 2**n)

def communication_complexity(circuit):
    return max(len(set(gate[1] for gate in layer)) for layer in circuit)

def run_trial(seed: int) -> dict:
```

```

random.seed(seed)
results = []
n_values = [5, 10, 15, 20, 30, 40]

for n in n_values:
    f = generate_random_boolean_function(n)
    circuit = construct_circuit(f)
    min_d_f = p_adic_divergence(f)
    c_f = communication_complexity(circuit)
    results.append((min_d_f, c_f))

if not results:
    return {
        "metric_name": "p-adic Divergence vs Communication Complexity",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

min_d_values, c_values = zip(*results)
correlation_coefficient = sum((min_d - sum(min_d_values) / len(min_d_values)) * (c - sum(c_values) / len(c_values))
                             for min_d, c in zip(min_d_values, c_values)) / (len(results) * sum(min_d_values * c_values))
mean_difference = abs(sum(min_d - c for min_d, c in zip(min_d_values, c_values)) / len(results))

return {
    "metric_name": "p-adic Divergence vs Communication Complexity",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": correlation_coefficient >= 0.7 and mean_difference <= 3,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_dev = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its intended task of verifying the conjecture. | next: Re-run the test with increased time limits or consider alternative methods to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14293
2	preregistration	ollama_remote	glm4:latest	0	0	14841
3	novelty	ollama_remote	glm4:latest	0	0	14554
4	novelty	ollama_remote	glm4:latest	0	0	8247
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15495
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42877
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	46438
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	36175
9	judge	ollama_remote	glm4:latest	0	0	25844

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 218765 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e29119440ded.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e29119440ded.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e29119440ded.tar.gz` (if generated)